

第六章习题参考答案

包含题目：习题 6.3 – 6.5 以及 6.7 – 6.17

题目 6.3 将下列二进制数转化为十进制数，假设此二进制数分别为原码、反码和补码整数。

- (1) 0111
- (2) 1110
- (3) 11111111
- (4) 10000000

解答

- (1) 若是原码，则为 7；若是反码，则为 7；若是补码，则为 7。
- (2) 若是原码，则为 -6；若是反码，则为 -1；若是补码，则为 -2。
- (3) 若是原码，则为 -127；若是反码，则为 0；若是补码，则为 -1。
- (4) 若是原码，则为 -0；若是反码，则为 -127；若是补码，则为 -128。

题目 6.4 将下列十进制数分别转化为 8 位二进制原码、反码和补码整数。

- (1) -86
- (2) 85
- (3) -127
- (4) 127

解答

- (1) 原码 1101 0110，反码 1010 1001，补码 1010 1010。
- (2) 原码 0101 0101，反码 0101 0101，补码 0101 0101。
- (3) 原码 1111 1111，反码 1000 0000，补码 1000 0001。
- (4) 原码 0111 1111，反码 0111 1111，补码 0111 1111。

题目 6.5 如果二进制补码整数最后一位是 0，表明该数是偶数，如果最后两位是 00，则表明该数有什么特点？

解答 能被 4 整除

题目 6.7 对于一个二进制数，如果向右移一位，则意味着进行了什么运算？

解答 除以 2

题目 6.8 做下列二进制补码整数加法运算，给出十进制形式的结果，并判断是否产生溢出。

(1) $1101 + 01010101$

(2) $0111 + 0101$

(3) $11111111 + 01$

(4) $01 + 1110$

(5) $0111 + 0001$

(6) $1000 + 11$

(7) $1100 + 00110011$

(8) $1010 + 101$

解答

(1) $11111101 + 01010101 = (1)01010010 = 82$ ，没有溢出。

(2) $0111 + 0101 = 1101 = -3$ ，溢出。

(3) $11111111 + 00000001 = (1)00000000 = 0$ ，没有溢出。

(4) $0001 + 1110 = 1111 = -1$ ，没有溢出。

(5) $0111 + 0001 = 1000 = -8$ ，溢出。

(6) $1000 + 1111 = (1)0111 = 7$ ，溢出。

(7) $11111100 + 00110011 = (1)00101111 = 47$ ，没有溢出。

(8) $1010 + 1101 = (1)0111 = 7$ ，溢出。

题目 6.9 做下列二进制数逻辑运算，结果以二进制形式给出。

(1) $11001100 \text{ AND } 01010101$

(2) $(1100 \text{ AND } 0101) \text{ AND } 1101$

(3) $1100 \text{ AND } (0101 \text{ AND } 1101)$

(4) $11001100 \text{ OR } 01010101$

(5) $(1100 \text{ OR } 0101) \text{ OR } 1101$

- (6) $1100 \text{ OR } (0101 \text{ OR } 1101)$
- (7) $\text{NOT } (\text{NOT } 1011)$
- (8) $1101 \text{ XOR } 0101$
- (9) $\text{NOT} ((\text{NOT } 1101) \text{ OR } (\text{NOT } 0101))$
- (10) $\text{NOT} ((\text{NOT } 1101) \text{ AND } (\text{NOT } 0101))$
- (11) $((\text{NOT } 1101) \text{ AND } 0101) \text{ OR } (1101 \text{ AND } (\text{NOT } 0101))$

解答

- (1) 0101 0111
- (2) 0000
- (3) 0000
- (4) 1101 0111
- (5) 1111
- (6) 1111
- (7) 0111
- (8) 0111
- (9) 1101
- (10) 0110

题目 6.10 给出下列十进制数的 IEEE 浮点数表示形式 (32 位), 并将这些数的 IEEE 浮点数转换为十六进制表示。

- (1) 32.9375
- (2) $-32\frac{45}{128}$
- (3) -2^{-140}
- (4) 65536

解答

- (1) 0 1000 0100 0000 0111 1000 0000 0000 000, 十六进制为 0x4203C000
- (2) 1 1000 0100 0000 0010 1101 0000 0000 000, 十六进制为 0xC2016800
- (3) 1 0000 0000 0000 0000 0000 0100 0000 000, 十六进制为 0x80000200

(4) 0 1000 1111 0000 0000 0000 0000 000, 十六进制为 0x47800000

题目 6.11 给出下列 IEEE 浮点数的十进制数表示形式。

(1) 0 00000001 000000000000000000000000

(2) 0 00000000 000000001000000000000000

(3) 1 11111011 000000000000000000000000

(4) 1 10000001 101010000000000000000000

(5) 0 01111101 010101000000000000000000

解答

(1) 2^{-126}

(2) 2^{-136}

(3) -2^{124}

(4) -6.625

(5) $(0.01010101)_2 = 0.33203125$

题目 6.12 将下列 16 位的二进制补码整数的十六进制数转换为十进制数。

(1) x8000

(2) x7FFF

(3) x1234

(4) xABCD

解答

(1) $1000\ 0000\ 0000\ 0000 = -2^{15}$

(2) $0111\ 1111\ 1111\ 1111 = 2^{15} - 1$

(3) $0001\ 0010\ 0011\ 0100 = 4660$

(4) $1010\ 1011\ 1100\ 1101 = -21555$

题目 6.13 将下列十进制数转换为 16 位的二进制补码整数的十六进制表示。

(1) -86

(2) 85

(3) -127

(4) 127

解答

(1) 1111 1111 1010 1010, 十六进制为 0xFFAA

(2) 0000 0000 0101 0101, 十六进制为 0x0055

(3) 1111 1111 1000 0001, 十六进制为 0xFF81

(4) 0000 0000 0111 1111, 十六进制为 0x007F

题目 6.14 如下代码将分别输出哪此内容?

(1) `printf ("%c\n", 13 + 'A');`

(2) `printf ("%x\n", 130);`

解答

(1) N (换行)

(2) 82 (换行)

题目 6.15 解释如下代码段的作用。

```
1 char nextChar;
2 int x;
3
4 scanf ("%c", &nextChar);
5 printf ("%d\n", nextChar);
6
7 scanf ("%d", &x);
8 printf ("%c\n", x);
```

解答 读入一个字符, 并输出其 ASCII 码值; 读入一个数字, 并输出 ASCII 码值为该数字的相应字符。

题目 6.16 描述如下代码段的作用及输出。

```
1 int i, j;
2 int count = 0;
3
4 scanf ("%d", &i);
5 for (j = 0; j < 32; j++){
6     if (i & (1 << j)){
```

```
7     count++;  
8 }  
9 }  
10  
11 printf("%d\n", count);
```

解答 输出输入数字二进制值中的 1 的个数。

题目 6.17 如下代码是否会造成无限循环?

```
1 int i = 1;  
2 while (i > 0)  
3     i++;
```

解答 不会，当i的值增大到整型的最大值时，再增加i的值会发生负溢出，结束循环。

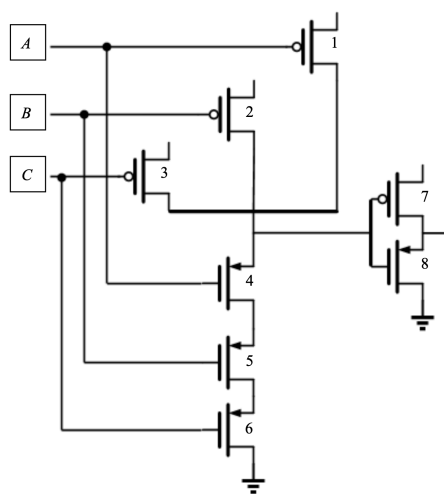
第七章习题参考答案

包含题目：习题 7.1 – 7.16

题目 7.1

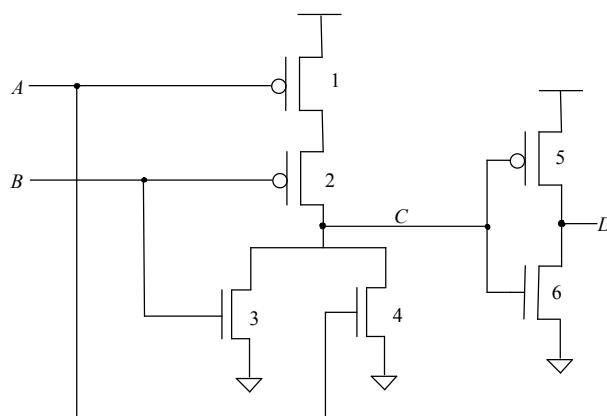
- (1) 分别画出 3 个输入的与门和 3 个输入的或门的晶体管级电路图。
- (2) 对于 $A = 0, B = 0, C = 1$ 的输入，分别在 3 个输入的与门和 3 个输入的或门晶体管级电路图中标出其表现。

解答 3 个输入的与门的晶体管级电路如下图所示



当 $A = 0, B = 0, C = 1$ 时，1/2/6/8 晶体管连通，最终输出 0。

3 个输入的或门的晶体管级电路如下图所示



当 $A = 0, B = 0, C = 1$ 时，1/2/6 晶体管连通，最终输出 1。

题目 7.2

- (1) 给出下图所示的晶体管级电路的真值表。

(2) 使用与、或、非门给出该真值表的门级电路图。

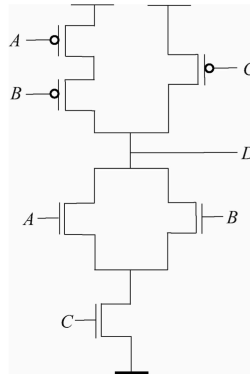


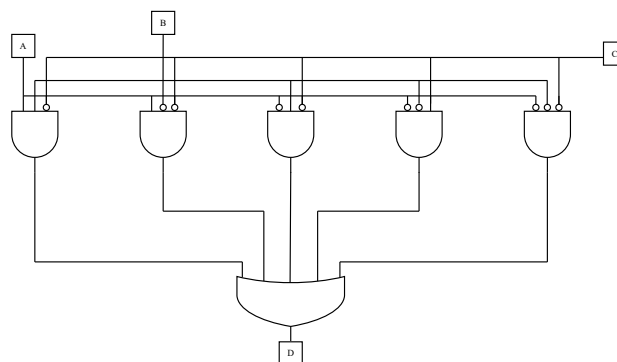
图 7.28 晶体管级电路

解答

(1) 如下表所示

A	B	C	D
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	1
0	0	1	1
0	0	0	1

(2) 如图所示



题目 7.3 下图所示的电路有一个缺陷，指出该缺陷。

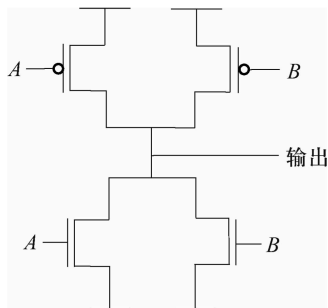
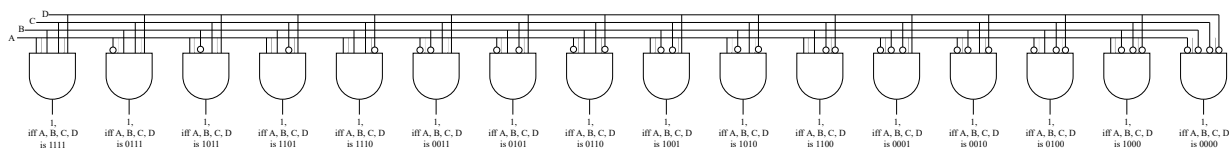


图 7.29 7.3 题电路

解答 若 $A = 0, B = 1$ 或 $A = 1, B = 0$ ，则同时接到了电源正极和大地负极，不能判断输出结果。

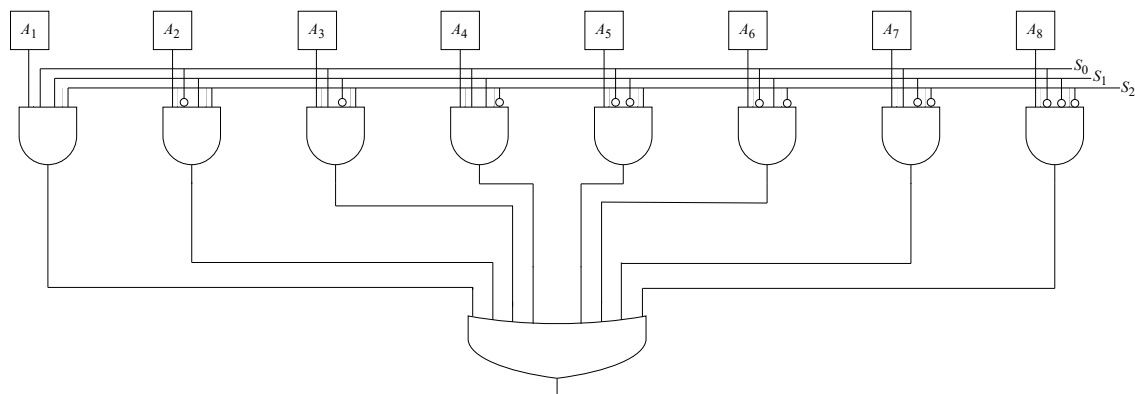
题目 7.4 画出有 4 个输入的译码器的门级电路图，并注明各输出为 1 的条件。

解答



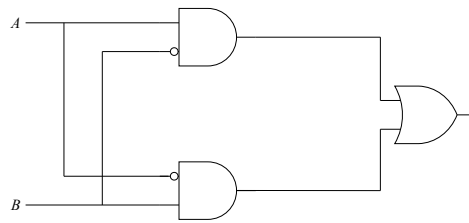
题目 7.5 画出有 8 个输入的多路选择器的门级电路图。

解答



题目 7.6 使用与、或、非门，给出异或函数的门级电路图。

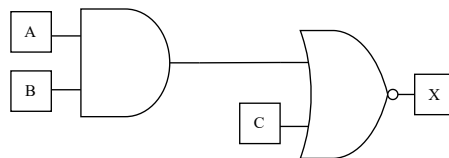
解答



题目 7.7 对于下表所示的真值表，使用 7.3 节（可编程逻辑阵列）给出的算法，生成其门级逻辑电路。

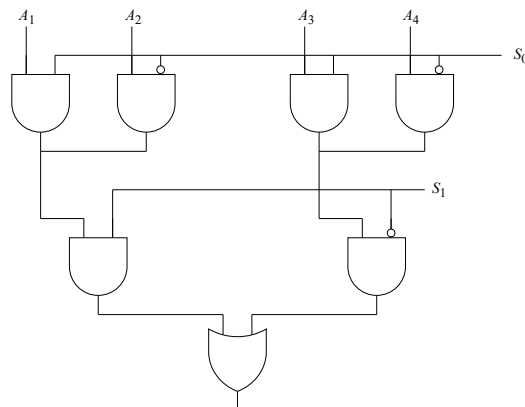
A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

解答 $X = \overline{A}BC + A\overline{B}C + AB\overline{C} = (\overline{A} + A\overline{B})C = \overline{AB} + C$



题目 7.8 只使用 2 选 1 的多路选择器，就可以实现 4 选 1 的多路选择器，给出其电路图。

解答



题目 7.9 根据下图所示的逻辑电路图，写出相应的真值表。

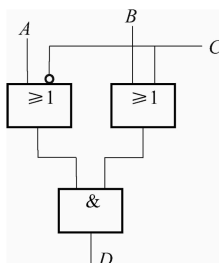


图 7.30 7.9 题逻辑电路图

解答 如下表所示

A	B	C	D
1	1	1	1
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	0
0	1	0	1
0	0	1	1
0	0	0	0

题目 7.10 下图中的每个矩形都表示一个全加法器，当 $X = 0$ 和 $X = 1$ 时，电路的输出分别是什么？在该电路图的基础上，构建一个可以实现加法/减法运算的逻辑电路图，使电路根据 X 的值，计算 $A + B$ 或 $A - B$ 的值。

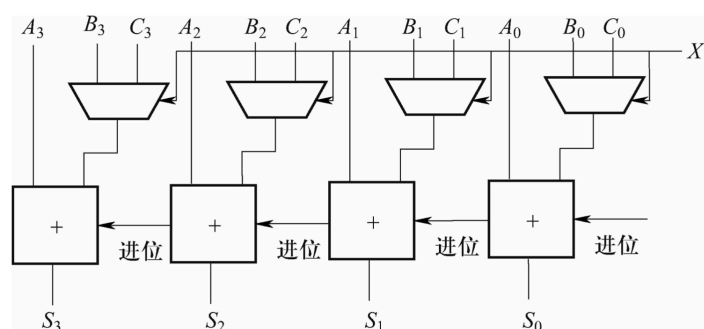
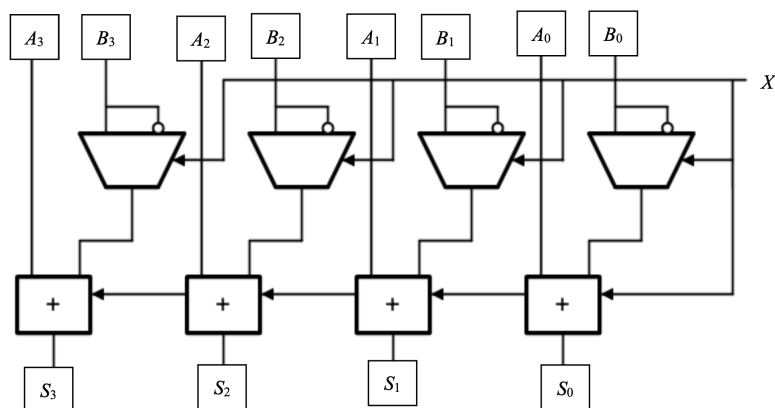


图 7.31 7.10 题电路

解答 当 $X = 0$ 时，输出分别为 $S_i = A_i + B_i + X$, $i = 0, 1, 2, 3$ ；当 $X = 1$ 时，输出分别为 $S_i = A_i + C_i + X$, $i = 0, 1, 2, 3$ 。电路图如下图所示



题目 7.11 一个逻辑结构的速度与从输入到达输出需传递经过的逻辑门的最长路径有关。假设与、或、非门都被计为一个门延迟，例如两个输入的译码器的传递延迟等于 2，这是因为有些输出需经过两个门的传递。

(1) 两个输入的多路选择器的传递延迟是多少？

(2) 1 位的全加法器的传递延迟是多少？

(3) 4 位的全加法器的传递延迟是多少？

(4) 32 位的全加法器的传递延迟是多少？

解答 (1) 3; (2) 3; (3) 12; (4) 96

题目 7.12 设计一个 1 位的比较器，该比较器电路有两个 1 位的输入 A 和 B ，有 3 个 1 位的输出 G (greater, 大于)、 E (equal, 等于) 和 L (less, 小于)。当 $A > B$ 时， G 为 1，否则 G 为 0；当 $A = B$ 时， E 为 1，否则 E 为 0；当 $A < B$ 时， L 为 1，否则 L 为 0。

(1) 给出此 1 位比较器的真值表。

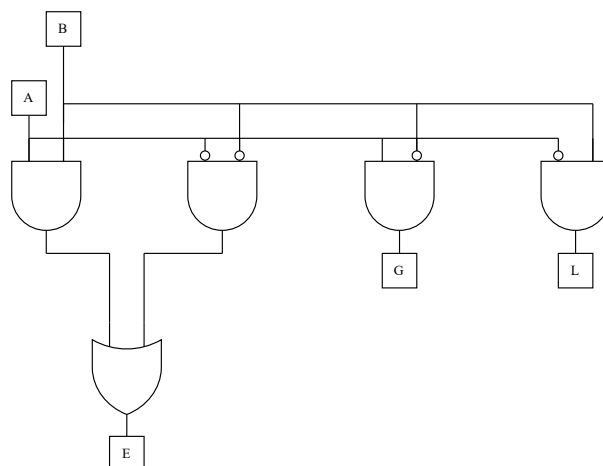
(2) 使用与、或、非门实现此比较器电路。

解答

(1) 如下表所示

A	B	G	E	L
1	1	0	1	0
1	0	1	0	0
0	1	0	0	1
0	0	0	1	0

(2) 如图所示



题目 7.13 参照下图，回答如下问题。

- (1) 当 S 和 R 都为 0 时，此逻辑电路的输出是什么？
- (2) 如果 S 从 0 转换到 1，输出是什么？
- (3) 此逻辑电路是存储元件吗？

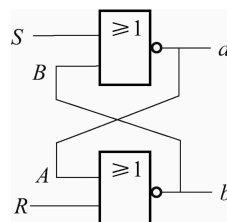


图 7.32 7.13 题电路

解答

- (1) 当 $S = 0, R = 0$ 时， $a = 0, b = 1$ 或 $a = 1, b = 0$
- (2) $a = 0, b = 1$
- (3) 是存储元件

题目 7.14 某个计算机有 4 个字节的寻址能力，访问其存储器的一个单元需要 64 位，该存储器的大小是多少（以字节为单位）？此存储器共存储多少位？

解答 2^{66} 字节； 2^{69} 位

题目 7.15 8 位被称为一个字节 (byte)，4 位被称为一个单元组 (nibble)。一个字节可寻址的存储器使用 14 位的地址，那么此存储器共存储了多少单元组？

解答 2^{15}

题目 7.16 对于下图所示的 4×2 位大小的存储器，回答以下问题。

- (1) 如果向单元 3 存储数值, $A[1:0]$ 和 WE 必须被设置为什么值?
- (2) 如果将此存储器的单元数目从 4 增加到 10, 需要多少条地址线? 存储器的寻址能力是否发生变化?

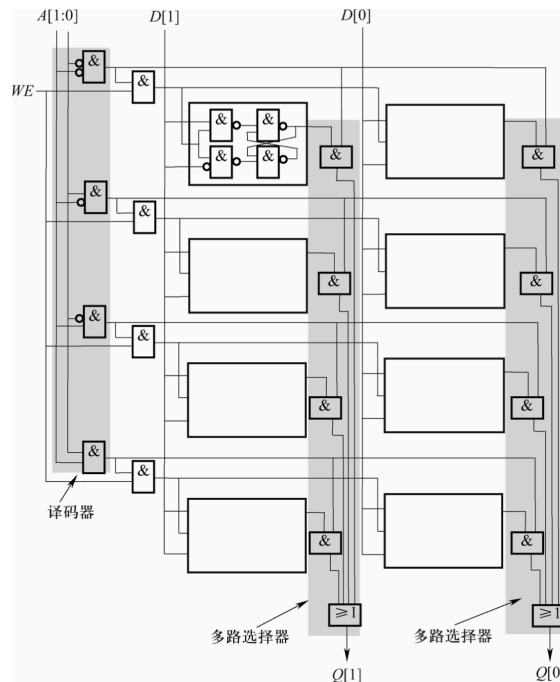


图 7.18 一个 4×2 位的存储器

解答

- (1) 如果单元从 0 起, $A[1:0]$ 为 11, 从单元 1 起则为 10; WE 必须被设置为 1
- (2) 需要 4 条地址线, 存储器的寻址能力不变

第八章习题参考答案

包含题目：习题 8.2 – 8.4

题目 8.2 下表显示了一个小的存储器的部分情况，根据此表回答以下问题。

- (1) 单元 0 和单元 4 包含的二进制数值分别是什么？
- (2) 每个单元内的二进制数值可以以不同的方式解释，如可以表示为无符号整数、补码整数、浮点数、ASCII 码等
 - I. 将单元 0 和单元 1 解释为 8 位补码整数，并以十进制形式写出结果。
 - II. 将单元 2 和单元 3 解释为 8 位无符号整数，并以十进制形式写出结果。
 - III. 将单元 4 解释为 ASCII 码值。
 - IV. 将单元 4、5、6 和 7 解释为一个 IEEE 浮点数（32 位），其中单元 4 包含该数的 [31:24] 位，单元 5 包含 [23:16] 位，单元 6 包含 [15:8] 位，单元 7 包含 [7:0] 位，以十进制形式写出结果
- (3) 存储单元的内容也可以是一条指令，将单元 8、9、10 和 11 解释为一条指令，其中单元 8 包含该指令的 [31:24] 位，单元 9 包含 [23:16] 位，单元 10 包含 [15:8] 位，单元 11 包含 [7:0] 位，该指令表示什么？
- (4) 一个二进制数值也可以被解释为一个存储单元的地址，如果存储在单元 11 中的数值是一个地址，它指的是哪个单元？该单元里包含的二进制数值是什么？

表 8.1 存储器情况

地址	数据	地址	数据
00000000	00000000	00000111	00000000
00000001	11111110	00001000	00000000
00000010	10000000	00001001	01100100
00000011	01111111	00001010	00101000
00000100	01000010	00001011	00000001
00000101	11010101	⋮	⋮
00000110	10000000		

解答

- (1) 0; 66
- (2)
 - I. 0; -2
 - II. 128; 127
 - III. B
 - IV. $0\ 10000101\ 101010110000000000000000 = 106.75$
- (3) 将 R3 和 R4 中的数相加，结果存到 R5 中

(4) 单元 11 中存储的是 00000001，即单元 1；那个单元中的二进制数值为 11111110，即 -2

题目 8.3 假设一个 16 位的指令采取如下图所示格式，如果共有 12 个操作码和 8 个寄存器，那么“补码整数”能够表示的数值范围是什么？

操作码	源寄存器	目标寄存器	补码整数
-----	------	-------	------

解答 $-2^5 \sim 2^5 - 1$ ，即 $-32 \sim 31$

题目 8.4 假设一个 32 位的指令采取如下图所示格式，如果共有 200 个操作码和 60 个寄存器，“无符号整数”能够表示的最大数是多少？

操作码	目标寄存器	源寄存器 1	源寄存器 2	无符号整数
-----	-------	--------	--------	-------

解答 $2^6 - 1 = 63$

第九章习题参考答案

包含题目：习题 9.1 – 9.5 以及 9.9 – 9.11

题目 9.1 对于 DLX 的 I-类型指令，试回答以下问题。

- (1) 立即数的范围是多少？
- (2) 如果重新定义 DLX 的 ISA，使得立即数表示无符号整数，那么，立即数的范围是多少？
- (3) 如果立即数表示无符号整数，那么执行位于 $x3000\ 0000 \sim x3000\ 0003$ 中的 LW 指令，该指令使用 R6 作为基址寄存器，且 R6 的值为 $x4000\ 0000$ ，那么，能够加载的数据的最大地址是多少？
- (4) 如果重新定义 DLX 的 ISA，将寄存器的数量从 32 个降低到 16 个，那么，在 I-类型的指令中能够表示的立即数的最大值是多少？假设立即数仍表示补码整数。

解答

- (1) $-2^{15} \sim 2^{15} - 1$
- (2) $0 \sim 2^{16} - 1$
- (3) $0x4000\ FFFF$
- (4) $2^{17} - 1$

题目 9.2 对于 DLX 的 R-类型指令，如果重新定义 DLX 的 ISA，将寄存器的数量从 32 个增加至 128 个，是否可行？

解答 在 R-类型指令中，有 5 个位没有使用。将寄存器的数量从 32 个增加到 128 个，所需位数加 2，总共需要位数加 6，而只有 5 个位可用，故不可行。

题目 9.3 假设某计算机的存储器包括 65 536 个单元，每个单元包含 16 位的内容，试回答以下问题。

- (1) 需要多少位表示地址？
- (2) 假设每条指令都由 16 位组成，其中一条指令与 DLX 无条件跳转指令（J 指令）工作机制类似。如果该指令位于单元 10 中，要跳转至地址 20，那么 PC 相对偏移量应为多少？

解答

- (1) 16 位
- (2) $20 - (10 + 1) = 9$

题目 9.4

- (1) 将 R8 乘以 8，并将结果存于 R9 中，使用一条 DLX 指令能够实现吗？

解答

- 可以, 相当于将 R8 左移 3 位后存入 R9 中, 相应的指令为 001101 01000 01001 0000000000000011
- 不可以, 左移 3 位可能导致高位溢出的问题

题目 9.5 试回答下列问题:

- (1) 使用一条 DLX 指令, 可以将 R1 中的值移至 R2 中吗?
- (2) 使用一条 DLX 指令, 可以将 R1 中的值按位取反吗?
- (3) 假设 R1 中存储的位组合的最右边两位有特殊的重要性, 根据这两位数值完成 4 个任务之一。使用一条 DLX 指令, 将这两位孤立出来。

解答

- (1) 可以, 使用如下指令中的任意一条均可满足: ORI R2, R1, #0; ANDI R2, R1, #-1; ADDI R2, R1, #0; ADD R2, R1, R0
- (2) 可以, 指令为 XORI r1, r1, #-1
- (3) ANDI R1, R1, #3

题目 9.9 如下表所示, 当一段起始于单元 x3000 0000 的程序执行结束后, R1~R6 的值分别是多少?

表 9.1 9.9 题存储器情况

地址	数据
x3000 0000	001100 00000 00001 0100 0000 0000 0000
x3000 0004	011100 00001 00010 0000 0000 0000 0100
x3000 0008	010110 00001 00011 0000 0000 0000 0100
x3000 000C	011101 00001 00010 0000 0000 0000 0000
x3000 0010	011100 00010 00100 0000 0000 0000 0100
x3000 0014	010110 00001 00101 0000 0000 0000 0001
x3000 0018	010111 00010 00100 0000 0000 0000 0011
x3000 001C	011100 00001 00110 0000 0000 0000 0000
x3000 0020	011100 00110 00110 0000 0000 0000 0000
x3000 0024	110000 00000 00000 0000 0000 0000 0000
⋮	⋮
x4000 0000	1000 0111 0110 0101 0100 0011 0010 0001
x4000 0004	0100 0011 0010 0001 0000 0000 0000 0000
⋮	⋮
x4321 0000	0000 0000 0000 0000 0100 0000 0000 0000
x4321 0004	0000 0000 0000 0000 1000 0000 0000 0000

解答 将机器代码翻译成汇编语言如下

地址	数据	解释
x3000 0000	001100 00000 00001 0100 0000 0000 0000	LHI R1, 0x4000
x3000 0004	011100 00001 00010 0000 0000 0000 0100	LW R2, 4(R1)
x3000 0008	010110 00001 00011 0000 0000 0000 0100	LBR3, 4(R1)
x3000 000C	011101 00001 00010 0000 0000 0000 0000	SW 0(R1), R2
x3000 0010	011100 00010 00100 0000 0000 0000 0100	LW R4, 4(R2)
x3000 0014	010110 00001 00101 0000 0000 0000 0001	LBR5, 1(R1)
x3000 0018	010111 00010 00100 0000 0000 0000 0011	SB 3(R2), R4
x3000 001C	011100 00001 00110 0000 0000 0000 0000	LW R6, 0(R1)
x3000 0020	011100 00110 00110 0000 0000 0000 0000	LW R6, 0(R6)
x3000 0024	110000 00000 00000 0000 0000 0000 0000	TRAP 0
-----	-----	-----
x4000 0000	1000 0111 0110 0101 0100 0011 0010 0001	0x87654321
x4000 0004	0100 0011 0010 0001 0000 0000 0000 0000	0x43210000
-----	-----	-----
x4321 0000	0000 0000 0000 0000 0100 0000 0000 0000	0x00004000
x4321 0004	0000 0000 0000 0000 1000 0000 0000 0000	0x00008000

R1 = 0x4000 0000, R2 = 0x4321 0000, R3 = 0x0000 0043, R4 = 0x0000 8000, R5 = 0x0000 0021, R6 = 0x0000 4000。

题目 9.10 下表显示了 DLX 存储器的一部分情况，如果条件分支将控制转移到 x3000 0000 单元，那么 R1 和 R2 有什么特点？

表 9.2 9.10 题存储器的一部分

地址	数据
x3000 0000	001011 00001 00001 1111 1111 1111 1111
x3000 0004	000000 00001 00010 00011 00000 000001
x3000 0008	001011 00011 00011 1111 1111 1111 1111
x3000 000C	101000 00011 00000 1111 1111 1111 0000

解答 将机器代码翻译成汇编语言如下

```

1 XORI r1, r1, #-1;
2 ADD r3, r1, r2;
3 XORI r3, r3, #-1;
4 BEQZ r3, 0xFFFF0;
```

由于条件分支将控制转移到 x3000 0000 单元，因此 $r3 = 0$ ，即 $r3 = \sim(\sim r1 + r2) = 0$ ，所以 $\sim r1 + r2 = -1$ ， $-r2 = \sim r1 + 1 = \sim r2 + 1$ ，因此有 $r1 = r2$ 。

题目 9.11 如果在下表所示的 DLX 指令序列执行结束时，R1 中存储的值为 7，由此可推知 R2 的什么信息？

表 9.3 9.11 题存储器情况

地址	数据
x3000 0000	000001 00000 00001 0000 0000 0000 0000
x3000 0004	000001 00000 00011 0000 0000 0001 0000
x3000 0008	000001 00000 00100 0000 0000 0000 0001
x3000 000C	000000 00010 00100 00101 00000 001001
x3000 0010	101000 00101 00000 0000 0000 0000 0100
x3000 0014	000001 00001 00001 0000 0000 0000 0001
x3000 0018	001101 00100 00100 0000 0000 0000 0001
x3000 001C	000011 00011 00011 0000 0000 0000 0001
x3000 0020	101001 00011 00000 1111 1111 1110 1000
x3000 0024	110000 00000 00000 0000 0000 0000 0000

解答 将机器代码翻译成汇编语言如下

地址	数据	解释
x3000 0000	000001 00000 00001 0000 0000 0000 0000	ADDI R1, R0, #0
x3000 0004	000001 00000 00011 0000 0000 0001 0000	ADDI R3, R0, 0x10
x3000 0008	000001 00000 00100 0000 0000 0000 0001	ADDI R4, R0, #1
x3000 000C	000000 00010 00100 00101 00000 001001	AND R5, R2, R4
x3000 0010	101000 00101 00000 0000 0000 0000 0100	BEQZ R5, 0x04
x3000 0014	000001 00001 00001 0000 0000 0000 0001	ADDI R1, R1, #1
x3000 0018	001101 00100 00100 0000 0000 0000 0001	SLLI R4, R4, #1
x3000 001C	000011 00011 00011 0000 0000 0000 0001	SUBI R3, R3, #1
x3000 0020	101001 00011 00000 1111 1111 1110 1000	BNEZ R3, 0xFFE8
x3000 0024	110000 00000 00000 0000 0000 0000 0000	TRAP 0

对应的 C 语言代码为

```

1  R1 = 0;
2  R3 = 16;
3  R4 = 1;
4  do{
5      R5 = R2 & R4;
6      if (R5 != 0){
7          R1 = R1 + 1;
8      }
9      R4 = R4 * 2;
10     R3 = R3 - 1;
11 }while (R3 != 0);

```

此程序检测 R2 的二进制形式中 1 的个数。由 R1 为 7 可以知道 R2 的二进制形式的后 16 位中有 7 个 1 和 $16 - 7 = 9$ 个 0。

第十章习题参考答案

包含题目：习题 10.1 和 10.2

题目 10.1 下表中的 DLX 程序实现了什么？

表 10.1 10.1 题 DLX 程序

x4000 0000	001100 00000 00001 0011 0000 0000 0000
x4000 0004	000001 00001 00010 0000 0000 0001 0100
x4000 0008	000001 00000 00011 0000 0000 0000 0101
x4000 000C	011100 00001 00100 0000 0000 0000 0000
x4000 0010	011100 00010 00101 0000 0000 0000 0000
x4000 0014	000000 00100 00101 00100 00000 000001
x4000 0018	011101 00001 00100 0000 0000 0000 0000
x4000 001C	000001 00001 00001 0000 0000 0000 0100
x4000 0020	000001 00010 00010 0000 0000 0000 0100
x4000 0024	000011 00011 00011 0000 0000 0000 0001
x4000 0028	101001 00011 00000 1111 1111 1110 0000
x4000 002C	110000 00000 00000 0000 0000 0000 0000

解答 将机器语言指令翻译成汇编代码如下

```

1 LHI R1, 0x3000
2 ADDI R2, R1, 0x14
3 ADDI R3, R0, 0x5
4 LW R4, R1
5 LW R5, R2
6 ADD R4, R5, R4
7 SW R4, R1
8 ADDI R1, R1, 0x4
9 ADDI R2, R2, 0x4
10 SUBI R3, R3, 0x1
11 BNEZ R3, 0xFEE0
12 TRAP #0

```

该段程序的作用是将 0x3000 0000 到 0x 3000 0013 存储的 5 个整数分别与存储在 0x3000 0014 到 0x3000 0028 的 5 个整数分别相加，并将结果写入到前者对应的地址。

题目 10.2 对于下表所示的机器语言程序，R1 的初始值是什么时，可以使得 R2 的最终值为 7？

表 10.2 10.2 题机器语言程序

x3000 0000	001100 00000 00011 1000 0000 0000 0000
x3000 0004	000001 00000 00010 0000 0000 0000 0000
x3000 0008	000000 00011 00001 00100 00000 001001
x3000 000C	101000 00100 00000 0000 0000 0000 0100
x3000 0010	000001 00010 00010 0000 0000 0000 0001
x3000 0014	000000 00001 00001 00001 00000 000001
x3000 0018	101000 00001 00000 0000 0000 0000 0100
x3000 001C	101100 11111 11111 1111 1111 1110 1000
x3000 0020	110000 00000 00000 0000 0000 0000 0000

解答 将机器语言指令翻译成汇编代码如下

```

1 LHI R3, R0, 0x8000
2 ADDI R2, R0, 0x0
3 AND R4, R3, R1
4 BEQZ R4, 0x4
5 ADDI R2, R2, 0x1
6 ADD R1, R1, R1
7 BEQZ R1, 0x4
8 J 0xFFFE8
9 TRAP #0

```

当 R1 的内容为 0xFE00 0000 时, R2 的最终值为 7。

第十一章习题参考答案

包含题目：习题 11.8、11.10、11.12、11.14、11.16 和 11.17

题目 11.8 对于如下程序

(1) 构建符号表

(2) 该程序实现了什么

```

                                . data      x00007000
SaveR2:                        . space      4
SaveR4:                        . space      4
HelloWorld:                    . asciiz     " Hello, World!"
;

                                . text      x40000000
                                . global     main
main:                          SW           SaveR2( R0 ), r2
                                SW           SaveR4( R0 ), r4
;

                                ADDI         R2, R0, HelloWorld
LOOP:                          LB           R4, 0( R2 )
                                BEQZ         R4, Return
                                TRAP         x07
                                ADDI         R2, R2, #1
                                J            LOOP
;
Return:                        LW           R2, SaveR2( R0 )
                                LW           R4, SaveR4( R0 )
                                TRAP         x00

```

解答

(1) 构建符号表：

符号	地址
SaveR2	x0000 7000
SaveR4	x0000 7004
HelloWorld	x0000 7008
main	x4000 0000
LOOP	x4000 000C
Return	x4000 0020

(2) 该程序在不改变 R2 和 R4 值的情况下，输出“Hello, World!”。

题目 11.10 对于如下程序

(1) 将该程序翻译为机器语言程序。

(2) 假设在这个程序执行之前，在 NUM1 中设置了一个正整数的值，该程序实现了什么？

```

                                . data      x30000000
NUM1:                          . space     4
NUM2:                          . space     4
                                ;

                                . text      x40000000
                                . global    main
main:                          ADDI        R1,R0,#0
                                LW          R2,NUM1(R0)
                                ANDI        R3,R2,#1
                                BNEZ        R3,NEXT
                                SUBI        R2,R2,#1
NEXT:                          ADD         R1,R1,R2
                                SUBI        R2,R2,#2
                                SLEI        R3,R2,#0
                                BEQZ        R3,NEXT
                                SW          NUM2(R0),R1
                                TRAP        x00

```

解答

(1) 第一步，建立符号表

符号	地址
NUM1	x3000 0000
NUM2	x3000 0004
main	x4000 0000
NEXT	x4000 0014

第二步，翻译为机器语言

地址	数据	解释
x4000 0000	000001 00000 00001 0000 0000 0000 0000	ADDI R1, R0, #0
x4000 0004	001100 00000 00100 0011 0000 0000 0000	LHI R4, x3000
x4000 0008	011100 00100 00010 0000 0000 0000 0000	LW R2, x0000(R4)
x4000 000C	001001 00010 00011 0000 0000 0000 0001	ANDI R3, R2, #1
x4000 0010	101001 00011 00000 0000 0000 0000 0100	BNEZ R3, NEXT
x4000 0014	000011 00010 00010 0000 0000 0000 0001	SUBI R2, R2, #1
x4000 0018	000000 00001 00010 00001 00000 000001	ADD R1, R1, R2
x4000 001C	000011 00010 00010 0000 0000 0000 0010	SUBI R2, R2, #2
x4000 0020	010010 00010 00011 0000 0000 0000 0000	SLEI R3, R2, #0
x4000 0024	101000 00011 00000 1111 1111 1111 0000	BEQZ R3, NEXT
x4000 0028	011101 00100 00001 0000 0000 0000 0100	SW x0004(R4), R1
x4000 002C	110000 00000 00000 0000 0000 0000 0000	TRAP x00

(2) 该程序将不大于 NUM1 中的所有奇数求和，存入 NUM2 中。

题目 11.12 如下程序存在一个错误，指出该错误并进行修复。该错误可以在汇编时还是在运行时被检测出来？

```

                .data          x30000000
NUM:            .space        4
;
                .text          x40000000
                .global        main
main:           ADDI           R1,R0,#100000
                SW             NUM(R0),R1
                TRAP            x00

```

解答 该程序的错误在 `main: ADDI R1, R0, #100000` 中，I-型 DLX 指令中的立即数为 16 位补码整数，能表示的十进制范围为 $-2^{15} \sim 2^{15} - 1$ ，即 $-32768 \sim 32767$ ，100000 在此范围之外。

要修复该错误，可以将该指令替换为 `LHI R2, x0001; ADDI R1, R2, x86A0` 或 `ADDI R1, R0, #20000; ADDI R1, R0, #20000`

由于汇编时立即数溢出，可知该错误是在汇编时被检测出来的。

题目 11.14 如下 DLX 程序用于判断一个字符串是否是“回文”（正向读和反向读都相同的字符串），例如，`strts` 就是回文。假设字符串起始于存储单元 `x30000000`。如果该字符串是回文，程序以 `R1` 的值为 1 结束，否则 `R1` 为 0。填空，将程序补充完整。

	.data	x30000000
STRING:	.ASCIIZ	"string"
;		
	.text	x40000000
	.global	main
main:	ADDI	R1,R0,#0
	ADDI	R2,R0,STRING
	ADDI	R3,R2,#0
LOOP1:	LB	R4,0(R2)
	BEQZ	R4,NEXT
	ADDI	R2,R2,#1
	J	LOOP1
NEXT:		
LOOP2:	LB	R4,0(R2)
	SEQ	R6,R4,R5
	BEQZ	R6,EXIT
	SLE	R6,R2,R3
	BEQZ	R6,LOOP2
	ADDI	R1,R1,#1
EXIT:	TRAP	x00

解答 填入的代码依次为:

```

1  LB  R5, 0(R3)
2  ADDI R2, R2, #1
3  ADDI R3, R3, #1
4  ADDI R1, R1, #1 或 ADDI R1, R6, #0

```

题目 11.16 使用 DLX 的 C 编译器对如下局部变量声明进行编译, 写出 DLX 汇编代码。

```

1  char c = 'c';
2  int x = 5;
3  int y;

```

解答

```

1  ADDI R5, R0, x63 或 ADDI R5, R0, #99
2  SW  -4(R30), R5
3  ADDI R5, R0, #5
4  SW  -8(R30), R5
5  ADDI R5, R0, #0
6  SW  -12(R30), R5

```

题目 11.17 使用 DLX 的 C 编译器对如下 switch 语句进行编译, 写出 DLX 汇编代码。

```

1  int i, j, k, x;

```

```
2  ...
3  switch (i){
4      case 0:
5          x = k + j;
6          break;
7      case 1:
8          x = k - j;
9          break;
10     case 2:
11         x = i + k;
12         break;
13     default:
14         x = i - k;
15 }
```

解答

```
1      ADDI R16, R0, #0; i
2      ADDI R17, R0, #0; j
3      ADDI R18, R0, #0; k
4      ADDI R19, R0, #0; x
5      .....
6      SEQI R1, R16, #0
7      BNEZ R1, Case_0
8      SEQI R1, R16, #1
9      BNEZ R1, Case_1
10     SEQI R1, R16, #2
11     BNEZ R1, Case_2
12     J Case_default
13 Case_0:  ADD R19, R18, R17
14         J Done
15 Case_1:  SUB R19, R18, R17
16         J Done
17 Case_2:  ADD R19, R16, R18
18         J Done
19 Case_default: SUB R19, R16, R18
20         J Done
21 Done:   .....
```

第十二章习题参考答案

包含题目：习题 12.6 – 12.8

题目 12.6 重新设计 DLX 的键盘输入处理程序：使用一个可读/写键盘数据和状态寄存器 (KBDSR) 代替 KBDR 和 KBSR, 且将内存地址 xFFFF 0010 ~ xFFFF 0013 分配给 KBDSR。KBDSR 中包含与 KBDR 相同的数据，即从键盘输入的 ASCII 码。那么，输入例程只需检查 KBDSR 是否有非零值出现，然后读取这个值，当读出该值后，硬件电路将 KBDSR 清零。参考 12.2 节，给出新的输入例程。

解答

```

1  A:      .word   xFFFF0010   ; KBDSR的起始地址
2          .....
3          LW      R1, A(R0)
4  START: LW      R2, 0(R1)    ; 测试是否有字符被输入
5          ADNI    R3, R2, #-1
6          BEQZ    R3, START
7          LW      R4, 0(R1)
8          J       NEXT_TASK   ; 执行下一个任务

```

题目 12.7 重新设计 DLX 的输入/输出处理程序：使用一个输入/输出状态寄存器 (IOSR), 代替 KBSR 和 DSR。IOSR[1] 是键盘设备就绪位，IOSR[0] 是显示器设备就绪位，且将内存地址 xFFFF 0010 ~ xFFFF 0013 分配给 IOSR。参考 12.2 节和 12.3 节，给出新的输入例程和输出例程。

解答 输入例程

```

1  A:      .word   xFFFF0010   ; IOSR的起始地址
2  B:      .word   xFFFF0004   ; KBDR的起始地址
3          .....
4          LW      R1, A(R0)
5  START: LW      R2, 0(R1)    ; 测试是否有字符被输入
6          ANDI    R3, R2, #2
7          BEQZ    R3, START
8          LW      R1, B(R0)
9          LW      R4, 0(R1)
10         J       NEXT_TASK   ; 执行下一个任务

```

输出例程

```

1  A:      .word   xFFFF0010   ; IOSR的起始地址
2  B:      .word   xFFFF000C   ; DDR的起始地址
3          .....
4          LW      R1, A(R0)
5  START: LW      R2, 0(R1)    ; 测试输出寄存器是否就绪
6          ANDI    R3, R2, #1
7          BEQZ    R3, START
8          LW      R1, B(R0)
9          SW      0(R1), R4
10         J       NEXT_TASK   ; 执行下一个任务

```

题目 12.8 如下 DLX 程序实现了什么?

```
1      .data
2 DSR:  .word  xFFFF0008
3 DDR:  .word  xFFFF000C
4      ;
5      .text
6      ADDI   R1, R0, x61
7      ADDI   R2, R0, #26
8 LOOP: LW     R3, DSR(R0)
9 START: LW    R4, 0(R3)
10     ADDI   R5, R4, #1
11     BEQZ   R5, START
12     LW     R3, DDR(R0)
13     SW     0(R3), R1
14     ADDI   R1, R1, #1
15     SUBI   R2, R2, #1
16     BNEZ   R2, LOOP
17     J      NEXT_TASK
```

解答 将上述程序写成伪代码

```
1  r1 = 'a'
2  r2 = 26
3  do{
4      print r1
5      r1 = r1 + 1
6      r2 = r2 - 1
7  }while (r2 != 0);
```

本程序实现了输出从“a”到“z”这 26 个字母。

第十三章习题参考答案

包含题目：习题 13.3、13.4、13.5、13.7、13.10、13.13 和 13.14

题目 13.3 设计一个新的 TRAP 服务例程，该服务例程读入一个字符并回显到屏幕上，如下所示：

```
1      .data    x00003C00
2 SaveR31:    .space 4
3 ;
4      .text    x0000 3D00
5      sw      SaveR31(r0), r31
6      getc
7      out
8 ;
9      lw      r31, SaveR31(r0)
10     jr      r31
```

- (1) 请说明调用此服务例程的指令。假设每个服务例程均占用 2^{10} 个单元。
- (2) 此服务例程能否正常运行？

解答

- (1) TRAP x08 的数据段地址是 x00002C00，这个服务例程的是 x00003C00，相差 2^{12} ，每个服务例程占有 2^{10} 个单元，所以该服务的调用指令为 TRAP x0C。
- (2)
 - 能够运行，getc 和 out 是指令助记符。
 - 不能够运行，没有对 R4 进行保存/恢复工作。

题目 13.4 重新定义 DLX 的 TRAP 机制，使其不使用 TRAP 向量表：

- (1) 通过将 TRAP 向量左移 8 位，形成相应服务例程的起始地址，那么，每个服务例程最多可以占用多少存储单元？256 个 TRAP 服务例程需要占用多少存储单元？
- (2) TRAP 向量提供的就是相应服务例程的起始地址，每个服务例程占用 2^{10} 个单元，那么，256 个 TRAP 服务例程需要占用多少存储单元？

解答

- (1) TRAP x08 对应的服务例程起始地址是 x00000800，TRAP x09 对应的服务例程起始地址是 x00000900，因此，每个服务例程最多有 2^8 个单元；256 个 TRAP 服务例程需要占用 2^{16} 个存储单元。
- (2) 需要占用 2^{18} 个存储单元

题目 13.5 执行如下程序，出现了一些异常行为，请解释原因。

```

1      .text      x4000 0000
2      .global    main
3 main: SW        24(R0), R0
4      GETC
5      OUT
6      TRAP       x00

```

解答 内存单元 x0000 0000 ~ x0000 03FF 被用来存储 TRAP 向量表，不能对其进行写入操作。

题目 13.7 执行如下汇编语言程序，输出是什么？

```

1      .data      x30000000
2 Char: .word      x61626364
3 HelloWorld: .asciiz "Hello, World!"
4      ;
5      .text      x40000000
6      .global    main
7 main: addi       r1, r0, #0
8      addi       r4, r0, Char
9      sw         4(r4), r1
10     trap       x08
11     ;
12 Exit: trap      x00

```

解答 屏幕输出“abcd”

题目 13.10 中断驱动的 I/O

(1). 如下程序实现了什么？

```

1      .data      x30000000
2 KBSR: .word      xFFFF0000
3      ;
4      .text      x40000000
5      .global    main
6 main: lw         r1, KBSR(r0)
7      lw         r2, 0(r1)
8      ori        r2, r2, #2
9      sw         0(r1), r2
10 LOOP: addi      r4, r0, x41
11      out
12      j         LOOP
13      trap      x00

```

(2). 如果有人键入了一个键，程序将被中断。修改图 13.11 给出的中断服务例程，对键盘的处理片段修改如下：

```

1  DEV1:  lw  r1, KBDR(r0)
2          lw  r4, 0(r1)
3          out                      ; 新增的内容
4          jr  r7

```

假设程序 (1) 开始执行，坐在键盘前的人键入了一个“8”，那么，屏幕上将显示什么内容？提示：“8”可于程序执行的任意时刻输入。

解答

- (1) 程序将 KBSR[1] 置 1，并且不断输出“A”。
- (2) 如果在原程序 out 指令前中断，则屏幕上将显示“A...A8A...A”；如果在原程序 out 指令后中断，则屏幕上将显示“A...A8A...A”。

题目 13.13 请写出处理如下任务的 C 语言的 I/O 函数调用。每个任务只需要一次调用。

- (1) 将一个电话号码按照 (XX)-XX-XXXXXXXX 的格式输出，假设电话号码被存储于 3 个整数变量中。
- (2) 读取一个 (XX)XX,XXXXXXXX 格式的电话号码到 3 个整数变量中。

解答

(1)

```

1  int first, second, third;
2  ...
3  printf ("The phone number is: (%d)-%d-%d\n", first, second, third);

```

(2)

```

1  int first, second, third;
2  scanf ("%d)%d,%d", &first, &second, &third);

```

题目 13.14 对于如下程序：

```

1  #include <stdio.h>
2
3  int main() {
4      int x = 0;
5      int y = 0;
6      char a = 'a';
7      char b = 'b';
8      a = getchar();
9      scanf ("%d%d", &x, &y);
10     b = getchar();

```



```
11 |  
12 |     printf ("%d %d %c %c", x, y, a, b);  
13 | }
```

- (1) 如果输入流是 10 20 C，程序的输出是什么？
- (2) 如果输入流是 C 10 20D，程序的输出是什么？
- (3) 如果输入流是 10 C，程序的输出是什么？

解答

- (1) 0 20 1 [空格]
- (2) 10 20 C D
- (3) 0 0 1 C

第十四章习题参考答案

包含题目：习题 14.7 – 14.10

题目 14.7 假设位于存储单元 NUM 中的值可以是任意一个大于 2 的 32 位的正整数。如下程序实现了什么？

```
1      .data    x30000000
2  NUM:      .word    #201
3  RESULT:   .space   #4
4  ;
5      .text    x40000000
6      .global  main
7  main:     addi     r1, r0, #0
8           addi     r2, r0, #2
9           lw       r3, NUM(r0)
10 ;
11 LOOP:     jal      MOD
12           beqz     r6, EXIT0
13           addi     r2, r2, #1
14           seq      r6, r2, r3
15           bnez     r6, EXIT1
16           j        LOOP
17 ;
18 EXIT1:     addi     r1, r1, #1
19 EXIT0:     sw       RESULT(r0), r1
20           trap     x00
21 ;
22 MOD:      addi     r4, r3, #0
23 AGAIN:     sub      r4, r4, r2
24           slei     r6, r4, #0
25           beqz     r6, AGAIN
26           jr       r31
```

解答 将上述汇编代码写成伪代码如下

```
1  r1 = 0; r2 = 2; r3 = NUM;
2  Do{
3      MOD()
4      If (r6 == 0)
5          break;
6      r2 = r2 + 1;
7      r6 = r2 == r3;
8      If (r6 == 1) {
9          r1 = r1 + 1;
10         break;
11     }
12 } while(1);
13 RESULT = r1;
14 MOD(){
```

```

15     r4 = r3;
16     do{
17         r4 = r4 - r2;
18         r6 = r4 <= 0;
19     } while(r6 == 0);
20 }

```

由于调用子例程 MOD 结束后 r6 始终是 1，所以不会从 EXIT0 直接退出，实现了 $r1 = 1$ 。

题目 14.8 假设从存储单元 DATA 开始存储的 10 个值可以是任意的 10 个 32 位的整数。如下程序实现了什么？

```

1      .data    x30000000
2  DATA:      .word    #3, #14, #35, #47, #5, #20, #12, #14, #6, #22
3  SaveR31:    .space   #4
4
5      .text    x40000000
6      .global  main
7  main:      addi     r1, r0, DATA
8             addi     r2, r0, #9
9  OutLoop:   beqz     r2, EXIT
10            addi     r3, r2, #0
11  InnerLoop: lw      r4, 0(r1)
12             lw      r5, 4(r1)
13             jal     CMP
14             addi     r1, r1, #4
15             subi     r3, r3, #1
16             bnez     r3, InnerLoop
17             slli     r6, r2, #2
18             sub      r1, r1, r6
19             subi     r2, r2, #1
20             j        OutLoop
21  EXIT:      trap     x00
22
23  CMP:       sw       SaveR31(r0), r31
24             slt      r6, r4, r5
25             bnez     r6, Return
26             jal     SWAP
27  Return:    lw       r31, SaveR31(r0)
28             ret
29
30  SWAP:      sw       4(r1), r4
31             sw       0(r1), r5
32             ret

```

解答 将上述代码翻译如下

```

1  r1 = [DATA]; r2 = 9;
2  While (r2 != 0){
3      r3 = r2;

```

```

4      Do{
5          r4 = r1[0];
6          r5 = r1[1];
7          CMP();
8          r1 = r1 + 4;
9          r3 = r3 - 1;
10     } while(r3 != 0);
11     r6 = r2 * 4;
12     r1 = r1 - r6;
13     r2 = r2 - 1;
14 }
15 CMP(){
16     r6 = r4 < r5;
17     if (r6 == 0){
18         SWAP();
19     }
20 }
21 SWAP(){
22     r1[1] = r4;
23     r1[0] = r5;
24 }

```

比较相邻两个数，前者不小于后者就交换，每次 OutLoop 选出一个最大数，实现排序。

题目 14.9 如下程序将存储单元 NUM 中的数值显示在屏幕上，但是不能正常工作。请找出原因并修复。

```

1      .data    x30000000
2 NUM:   .word   #8
3      ;
4      .text    x40000000
5      .global  main
6 main:   jal     SubA
7         trap   x07
8         trap   x00
9      ;
10 SubA:  lw      r1, NUM(r0)
11         jal     SubB
12         ret
13      ;
14 SubB:  addi    r4, r1, x30
15         ret

```

解答 嵌套子程序中上一层必须保存 r31，防止被下一层修改。可以通过在调用 jal SubB 之前保存 r31，调用之后恢复 r31 来修复。

题目 14.10 如下程序实现了什么？

```

1      .data    x30000000
2 STACK: .space #100

```

```

3 PROMPT: .ascii "Please enter your string: "
4 ;
5     .text    x40000000
6     .global  main
7 main:  addi   r4, r0, PROMPT
8        trap  x08
9        addi   r3, r0, STACK
10       addi   r5, r3, #100
11       addi   r29, r5, #0
12 Input: seq    r2, r29, r3
13        bnez   r2, Output
14        trap  x06
15        trap  x07
16        seqi   r2, r4, #10
17        bnez   r2, Output
18        jal    PUSH
19        j      Input
20 ;
21 Output: seq    r2, r29, r5
22         bnez   r2, DONE
23         jal    POP
24         trap  x07
25         j      Output
26 ;
27 DONE:  trap  x00
28 ;
29 PUSH:  subi   r29, r29, #4
30        sw     0(r29), r4
31        ret
32 ;
33 POP:   lw     r4, 0(r29)
34        addi   r29, r29, #4
35        ret

```

解答 将上述代码翻译如下

```

1 printf("Please enter your string:");
2 r3 = [STACK]; r5 = r3 + 100; r29 = r5;
3 While (1){
4     r2 = r29 == r3;
5     if (r2 == 1)
6         Output();
7     scanf();
8     printf();
9     r2 = r4 == 10;
10    if (r2 == 1)
11        Output();
12    PUSH();
13 }

```

```
14 Output(){
15     r2 = r29 == r5;
16     if (r2 == 1)
17         exit;
18     POP();
19     printf();
20     Output();
21 }
22 PUSH(){
23     r29 = r29 - 4;
24     r29[0] = r4;
25 }
26 POP(){
27     r4 = r29[0];
28     r29 = r29 + 4;
29 }
```

带回显地输入以换行符为结束标记的字符串（最长 25），再以反序输出到屏幕上。

第十五章习题参考答案

包含题目：习题 15.1、15.2、15.3、15.7 和 15.9

题目 15.1 如下程序的输出是什么?

```
1  #include <stdio.h>
2
3  int Sub (int x, int y);
4
5  int main () {
6      int x = 2;
7      int y = 3;
8      int z;
9
10     z = Sub (x, y);
11     x = Sub (y, z);
12     y = Sub (x, y);
13     printf ("%d %d %d", x, y, z);
14 }
15
16 int Sub (int y, int x) {
17     return y - x;
18 }
```

解答 该程序的输出为 4 1 -1

题目 15.2 如下程序的输出是什么?

```
1  #include <stdio.h>
2
3  int Multiply (int x, int y);
4  int z = 4;
5
6  int main () {
7      int x = 2;
8      int y = 3;
9      int z;
10
11     z = Multiply (x, y);
12     x = Multiply (y, z);
13     y = Multiply (x, z);
14     printf ("%d %d %d \n ", z, x, y);
15 }
16
17 int Multiply (int x, int y) {
18     return x * y * z;
19 }
```

解答 该程序的输出为 24 288 27648[换行]

题目 15.3 如下程序的输出是什么?

```

1  #include <stdio.h>
2
3  void Swap (int x, int y);
4
5  int main(){
6      int x = 1;
7      int y = 2;
8
9      printf ("x=%d, y=%d\n", x, y);
10
11     Swap(x, y);
12     printf ("x=%d, y=%d\n", x, y);
13 }
14
15 void Swap(int x, int y){
16     int temp;
17     temp = x;
18     x = y;
19     y = temp;
20     printf ("x=%d, y=%d\n", x, y);
21 }

```

解答 该程序的输出为:

```

x=1, y=2
x=2, y=1
x=1, y=2[换行]

```

题目 15.7 使用 DLX 的 C 编译器对函数 ToUpper 进行编译, 请写出 DLX 汇编代码。

```

1  /*ToUpper函数*/
2  /*如果参数是小写字母*/
3  /*返回其大写字母的ASCII码*/
4  char ToUpper (char inchar) {
5      char outchar;
6      if ('a' <= inchar && inchar <= 'z')
7          outchar = inchar - ('a' - 'A');
8      else
9          outchar = inchar;
10     return outchar;
11 }

```

解答

```

1  ToUpperR:    SLTI    R1, R4, #97 ; R4 saves inchar

```



```

2          BNEZ    R1, Else
3          SLEI    R1, R4, #122
4          BEQZ    R1, Else
5          SUBI    R2, R4, #32
6          J       Done
7 Else:
8          ADDI    R2, R4, #0 ; R2 saves outchar
9          J       Done
10 Done:
11         ret

```

题目 15.9 对于如下 C 代码，存在 bug，请找出并修复：

```

1  #include <stdio.h>
2
3  int main() {
4      int x = 1;
5      int y = 2;
6      x = Func1 (x, y);
7      y = Func2 (y);
8      printf ("x = %d y = %d\n", x, y);
9  }
10
11 int Func1 (int x) {
12     return x + y;
13 }
14
15 int Func2 (int x) {
16     int y;
17     return x - y;
18 }

```

解答

1. 函数未声明：需要在 main 之前添加 Func1 和 Func2 的声明

```

1  int Func1 (int);
2  int Func2 (int);

```

2. 函数 Func1 的参数个数不匹配

3. 函数用到了未声明的变量：修改函数 Func1 的参数结构，同时更新其声明。

```

1  int Func1 (int, int);
2  ...
3  int Func1 (int x, int y){...}

```

4. 函数的局部变量没有初始化。（运行时错误）局部变量使用前最好都要初始化，防止读取为初始化的局部变量。

```
1  int y = 0;
```

修改后的代码如下：

```
1  #include <stdio.h>
2
3  int Func1 (int, int);
4  int Func2 (int);
5
6  int main() {
7      int x = 1;
8      int y = 2;
9      x = Func1 (x, y);
10     y = Func2 (y);
11     printf ("x = %d y = %d\n", x, y);
12 }
13
14 int Func1 (int x, int y) {
15     return x + y;
16 }
17
18 int Func2 (int x) {
19     int y = 0;
20     return x - y;
21 }
```

第十六章习题参考答案

包含题目：习题 16.1、16.2、16.9、16.10 和 16.14

题目 16.1 对于如下程序：

```

1  #include <stdio.h>
2  int main() {
3      int x = 1;
4      int *ptr1;
5      int **ptr2;
6
7      ptr2 = &ptr1;
8      *ptr2 = &x;
9      **ptr2 = 2;
10
11     x++;
12     (*ptr1)++;
13     (**ptr2)++;
14
15     printf ("%d\n", x);
16 }
```

- (1) 程序的输出是什么？提示：ptr2 是一个指向指针的指针。
- (2) 请描述语句 (**ptr2)++; 执行之后的运行时栈中的内容。

解答

- (1) 5[空格][换行]
- (2) 如图所示

0x4000 00BC		
0x4000 00C0	0x4000 00C0(R16)	Ptr1
0x4000 00C4	0x4000 00C4(R17)	Ptr2
0x4000 00C8	0x0000 0005(#5)	x
0x4000 00CC	R30	
0x4000 00D0	R31	<-R30

题目 16.2 下列代码片段的输出是什么？

(1)

```

1  char ch[7] = "1a2b3c";
2  int i, s = 0;
```

```

3  for (i = 0; i < 6; i++) {
4      if (ch[i] >= '0' && ch[i] <= '9')
5          s = s + ch[i] - '0';
6      else
7          s = s + ch[i] - 'a';
8  }
9  printf ("%d\n", s);

```

(2)

```

1  char str[13] = "hello world!";
2  char *p;
3  p = str;
4  while (*p != ' ') {
5      printf ("%c", *p - 'a' + 'A');
6      p++;
7  }

```

解答

(1) 9[换行]

(2) HELLO

题目 16.9 将函数 StringLength 翻译为 DLX 汇编语言。

```

1  int StringLength (char string[]) {
2      int index = 0;
3      while (string[index] != '\0')
4          index = index + 1;
5      return index;
6  }

```

解答

```

1  StringLength:  subi    r29, r29, #4
2                  sw     0(r29), r16 ; 压入r16 (寄存器的保存)
3                  addi   r16, r0, #0 ; r16 <- index
4  LOOP:         addi   r9, r4, r16 ; 计算string[index]的地址值
5                  lb     r10, 0(r9) ; r10 <- string[index]
6                  beqz   r10, EXIT
7                  addi   r16, r16, #1; index = index + 1
8                  J      LOOP
9  EXIT:         addi   r2, r16, #0 ; return index
10                 lw     r16, 0(r29) ; r16出栈
11                 addi   r29, r29, #4
12                 ret

```

R8~R15 和 R24、R25 用于存放临时产生的值，R16~R23 用于存放局部变量。

题目 16.10 如下代码从键盘读入一个字符串，将其中的小写字母转换为大写字母后输出。代码中存在 bug，请找出并修复。

```
1  #include <stdio.h>
2
3  char *ToUpper (char *inchar);
4
5  int main () {
6      char str[10];
7      printf ("Enter a string : ");
8      scanf ("%s", str);
9      printf ("%s \n", ToUpper (str) );
10 }
11
12 char *ToUpper (char *inchar) {
13     char str[10];
14     int i = 0;
15     while (*(inchar + i) != '\0') {
16         if ('a' <= *(inchar + i) && *(inchar + i) <= 'z')
17             *(str + i) = *(inchar + i) - ('a' - 'A');
18         else
19             *(str + i) = *(inchar + i);
20         i++;
21     }
22     return str;
23 }
```

解答 该程序的 bug 为：ToUpper 函数中返回值是一个局部变量的指针，离开该函数后指针所指的内容即为非法；根据题意可知，返回值应该仍然是输入参数。

解决方案：将 `char str[10];` 改为 `char *str = inchar;` 即可。

题目 16.14 如果输入流为 `abc123 def`

对于如下函数调用，字符数组 `x` 的值将是什么？

```
1  scanf ("%s", x)
```

解答 字符数组 `x` 中的值未发生改变，如果 `x` 声明时为初始化，将会是垃圾数据。